

# RECALL

Open-Source Agent Memory Engine

*Qwen Cloud Hackathon · Track 1: MemoryAgent*

**Document:** Parts 4 & 5 — Market Differentiation Analysis and Novel Feature Design

**Audience:** Internal Engineering Team

**Status:** Hackathon Submission Strategy — Pre-Build Reference

**Deadline:** 9 July 2026 · Prize: \$7,000 cash + \$3,000 credits per track

## Purpose of this Document

This document serves as the internal reference for the Recall engineering team. It answers two critical questions before a single line of code is written: **(1) How is Recall different from every existing product in the market?** and **(2) What genuinely new, market-absent features should Recall introduce?** Every team member should read this document in full before starting their sprint.

## Table of Contents

### PART 4 Market Differentiation Analysis

- 4.1 What Already Exists in the Market
- 4.2 The Three Genuine Differentiators of Recall
- 4.3 Strategic Positioning Statement

### PART 5 New Untouched Features — Not in Any Existing Product

- 5.1 Memory Conflict Detection + Resolution Engine
- 5.2 Memory Provenance Graph
- 5.3 Predictive Memory Pre-fetching
- 5.4 Cross-Session Confidence Accumulation
- 5.5 Cross-Agent Memory Scoping — Permission Layer
- 5.6 Priority Build Recommendation

PART 4

# Market Differentiation Analysis

Before building any feature, the team must understand the competitive landscape with precision. This section maps what already exists in the market so Recall does not waste sprint cycles rebuilding solved problems — and then identifies the exact design choices that make Recall architecturally distinct from every current product.

## 4.1 What Already Exists in the Market

**WARNING:** The features below are industry-standard. Building them without differentiation will result in judges classifying Recall as a Mem0 clone. Do not claim these as novel in your README or demo.

The following table maps every feature that is commonly associated with a memory layer to the existing product that already provides it. Any Recall feature in this list must be treated as infrastructure — necessary, but not a differentiator.

| Feature                                                                  | Already Provided By               | Maturity          |
|--------------------------------------------------------------------------|-----------------------------------|-------------------|
| Three / four-tier memory model (Episodic, Semantic, Procedural, Working) | Mem0, Letta, CrewAI               | Industry standard |
| Auto-extracting facts from conversation transcripts                      | Mem0 — core feature               | Mature            |
| Temporal filtering (filter memories by when they happened)               | Zep                               | Mature            |
| Token-cost-aware retrieval (don't send what isn't needed)                | Cloudflare Agent Memory           | Emerging          |
| Agent self-editing its own long-term memory store                        | Letta (MemGPT) — flagship feature | Mature            |
| Open-source library + pluggable storage backends                         | Mem0 is fully open-source         | Mature            |
| LangGraph / LangChain native memory adapters                             | LangMem (official LangChain)      | Mature            |
| Role-based multi-agent memory with memory=True flag                      | CrewAI                            | Mature            |

Bottom line: these are table-stakes features for any memory system. Build them, but do not lead with them.

## 4.2 The Three Genuine Differentiators of Recall

After surveying Mem0, Zep, Letta, Cognee, Cloudflare Agent Memory, and LangMem in depth, three specific design decisions in Recall have no equivalent in any product currently on the market. These are the features that must be front and center in the README, architecture diagram, and demo video.

### Differentiator 1 — Mathematical Decay with Retrieval-Reinforced Strength

This is Recall's strongest moat. No existing product models memory strength as a continuously decaying numeric value that is also **boosted on retrieval**.

| Product | How It Handles Stale Memory                                                                   | Passively Decays? |
|---------|-----------------------------------------------------------------------------------------------|-------------------|
| Zep     | Temporal search — filter by time, but stale memories sit at equal weight forever              | No                |
| Mem0    | Additive-only — no deletion or decay; memories accumulate indefinitely                        | No                |
| Letta   | Agent must explicitly decide to delete — no passive forgetting mechanism                      | No                |
| Recall  | Strength decays via Ebbinghaus curve; boosts on every retrieval; soft-deletes below threshold | Yes               |

Technical detail:

- Per-tier decay rates: episodic decays in days, semantic in weeks, procedural almost never
- Strength function:  $S(t) = S_{\text{max}} \times e^{(-\lambda t)}$ , where  $\lambda$  differs per tier
- Each retrieval multiplies current strength by a reinforcement factor  $r > 1$
- Memories below threshold  $\tau$  are tombstoned (soft-deleted) — not hard-deleted, preserving audit trails
- This is the only system where irrelevant memories naturally fade without an agent or human explicitly deleting them

**Benchmark angle:** Run a suite of 50 multi-session test cases with deliberately injected stale memories. Compare: (a) naive flat memory retrieval, (b) Mem0-style additive retrieval, (c) Recall decay-based retrieval. Metric: signal-to-noise ratio in the retrieved context window.

Differentiator 2 — Automated Procedural Memory Extraction

Procedural memory is a recognized category in cognitive science and appears in several product descriptions — but no tool **automatically detects and extracts procedural patterns** from user behavior without requiring the agent or user to trigger it manually.

- **Letta's approach (closest competitor):** The agent can self-edit its own memory, which includes procedural entries — but only if the agent is explicitly programmed to recognize and write them. There is no passive detection.
- **Recall's approach:** The consolidation engine analyzes session history for repeated task sequences. When the same N-step workflow appears M times (configurable thresholds), it is automatically crystallized into a Tier 3 procedural memory unit — no agent code changes needed.
- **Practical impact:** An agent serving a developer user will, after several sessions, automatically "know" that this user always wants pale cyan ReportLab styling for PDFs — without ever being explicitly told.

**Benchmark angle:** Construct a dataset of sessions with 8 deliberately repeated 5-step workflows. Measure: (a) how many sessions before Recall auto-extracts the procedural unit, (b) accuracy of the extracted unit against ground truth.

Differentiator 3 — Context Budget as a Formal First-Class API Parameter

Cloudflare Agent Memory addresses token cost by filtering memories on a need-to-know basis. That is a **retrieval filter**. Recall's context-budget parameter is an **optimization algorithm** — a fundamentally different and more powerful abstraction.

| Concept   | Cloudflare Agent Memory                                   | Recall                                                           |
|-----------|-----------------------------------------------------------|------------------------------------------------------------------|
| Mechanism | Filter: don't return memories below a relevance threshold | Push: maximize information coverage within a stated token budget |

|               |                                                       |                                                              |
|---------------|-------------------------------------------------------|--------------------------------------------------------------|
| API contract  | Implicit — no budget parameter; threshold is internal | Explicit: caller passes token_budget=2000; Recall guarantees |
| Compression   | No — returns or drops whole memory units              | Yes — lower-priority memories are summarized to fit more s   |
| Measurability | Latency only                                          | Information coverage per token — quantifiable and benchm     |

**Benchmark angle:** Same 20-session conversation history, three retrieval strategies: naive top-K, Cloudflare-style threshold filter, Recall budget-packer. Metric: relevant facts retrieved per 1,000 tokens.

4.3 Strategic Positioning Statement

**STRATEGY:** Do not position Recall as 'a memory layer for agents' — that space is crowded. Position it as 'the memory layer that knows how to forget.' Forgetting is the unsolved problem. Every other tool is additive. Lead with that.

***"Recall is the first open-source memory engine that models memory as a living system — where memories decay, reinforce, and self-organize, rather than accumulate indefinitely."***

Use this statement verbatim as the opening line of the GitHub README, the first slide of the architecture deck, and the first 10 seconds of the demo video.

PART 5

New Untouched Features — Not in Any Existing Product

The five features in this section were verified against the full competitive landscape — Mem0, Zep, Letta, Cognee, Cloudflare Agent Memory, LangMem, CrewAI, and the Anthropic/OpenAI assistant SDKs. None of these features exist in any listed product in a meaningful form. Each one addresses a real production failure mode that current tools ignore.

Feature Overview at a Glance

| #   | Feature Name                          | Core Gap It Fills                                                      | Build Effort |
|-----|---------------------------------------|------------------------------------------------------------------------|--------------|
| 5.1 | Conflict Detection + Resolution       | Agents hold contradictory beliefs silently; no tool detects this       | High         |
| 5.2 | Memory Provenance Graph               | No tool tracks why a belief is held or its evidence chain              | High         |
| 5.3 | Predictive Memory Pre-fetching        | All tools are reactive; no tool pre-warms the retrieval cache          | Medium       |
| 5.4 | Cross-Session Confidence Accumulation | No tool models how confidence evolves with corroboration/contradiction | Medium       |
| 5.5 | Cross-Agent Memory Scoping            | No formal permission model for multi-agent persistent memory sharing   | Medium       |

5.1 Memory Conflict Detection + Resolution Engine

The Gap

All current memory tools are additive. When a new session produces a belief that contradicts an existing one, every tool silently overwrites (last-write-wins) or appends a duplicate. The result is an agent that holds contradictory beliefs simultaneously and hallucinate confidently based on whichever memory lands in its context window first. This is a production failure mode with no solution in any listed product.

How It Works

- **On ingestion:** Every new Tier 2 semantic memory unit is checked for semantic similarity against all existing Tier 2 units using a cosine distance threshold
- **Conflict detection:** If two units are semantically close but factually opposite (e.g., 'User prefers Python' vs 'User is migrating to TypeScript'), a conflict event is raised
- **Qwen resolution:** Qwen-Max evaluates three signals — recency of each memory, confidence score of each, and surrounding context — and produces a resolution decision
- **Outcomes:** Auto-resolve with versioned update / flag for agent clarification on next session / merge into a nuanced composite belief (e.g., 'User uses Python for backend, exploring TypeScript for frontend')
- **Conflict history:** Every resolved conflict creates a permanent conflict log entry — the evolution of the belief is preserved, not just the current state

Example

Conflict Event Log (Example)

```

Session 12: extracted → 'User prefers Python' [confidence: 0.89]
Session 29: extracted → 'User is migrating everything to TypeScript' [confidence: 0.84]
Conflict raised: semantic distance = 0.12 (below threshold 0.25)
Qwen resolution: Session 29 is more recent; context indicates frontend work only
Resolved belief: 'User uses Python for backend services; TypeScript for frontend'
Status: auto-resolved | conflict_id: cflct_0041 | provenance updated

```

**Why this wins engineering judges:** This is not a UX feature — it is a correctness guarantee. A memory system with conflict detection is provably more reliable than one without. The conflict history log also provides a clean GDPR story: full audit trail of how a belief evolved.

**Benchmark:** Inject 20 deliberate contradictions across 60 sessions. Metric: auto-resolution accuracy vs. ground truth. Compare against last-write-wins baseline.

## 5.2 Memory Provenance Graph

### The Gap

Every memory tool stores *what* a belief says. None of them store *why* the system believes it, *how much evidence* supports it, or *how that evidence evolved* across sessions. This means agents cannot explain their own beliefs, and GDPR deletion is incomplete — deleting the memory entry does not delete the derived inferences built on top of it.

### How It Works

- Every memory unit carries a **provenance chain** — a lightweight directed acyclic graph recording its full derivation history
- **Direct statement:** User said 'I deploy on AWS' → creates a provenance node with type=direct, confidence=0.95
- **Corroboration:** Later session contains kubectl commands → adds a corroboration node; overall confidence increases
- **Contradiction:** User mentions a GCP trial → adds a contradiction node; conflict resolver runs; outcome logged in graph
- **Queryable:** Any agent or user can call GET /memory/{id}/why and receive the full evidence chain in plain language
- **GDPR hard delete:** DELETE /memory/{id}?cascade=true removes the memory unit and every provenance node derived from it

### Example Provenance Chain

```

"User deploys on AWS" [confidence: 0.91] [status: active]
■■■ session_14 → direct statement [weight: +0.35]
■■■ session_22 → corroboration via context [weight: +0.18]
■■■ session_31 → tool usage pattern (aws-cli) [weight: +0.22]
■■■ session_38 → contradiction: mentioned GCP trial [weight: -0.12]
■■■ conflict_resolved: AWS=primary, GCP=experimental [delta: -0.12 absorbed]

```

### Why This Is Architecturally Significant

Cognee uses knowledge graphs for data ingestion from unstructured text — but that is about representing *domain knowledge*. Recall's provenance graph represents the *epistemic history* of the agent's own beliefs about a user. These are entirely different problems. No other memory product has anything comparable to this.

**Benchmark:** Explainability score — given 30 memory units with known derivation histories, measure what percentage of provenance chains Recall correctly reconstructs when queried.

## 5.3 Predictive Memory Pre-fetching

### The Gap

Every memory retrieval system in the market is **reactive**: the agent sends a query, the memory layer fetches the relevant units, and the agent receives them. This model incurs retrieval latency on every turn — which compounds in long-horizon multi-step tasks. Nobody in the agent memory space has applied the systems-engineering principle of pre-fetching to this problem.

### How It Works

- **Intent classifier:** A lightweight Qwen-Plus call (cheap, low latency) runs on a sliding window of the last 3 conversation turns in parallel with the main agent
- **Cluster prediction:** The classifier outputs a predicted memory cluster tag (e.g., cloud-deployment, coding-preferences, past-errors)
- **Pre-fetch trigger:** Recall queries the memory store for all units matching the predicted cluster and stages them in an in-memory warm cache
- **Agent retrieval:** When the agent sends its formal memory query, the response is served from the warm cache — near-zero latency
- **Cache expiry:** Pre-fetched memories not consumed within the session TTL are discarded — no persistent side effects
- **Hit rate tracking:** Every pre-fetch records whether it was actually consumed — this drives adaptive threshold tuning

### The Systems Engineering Analogy

This pattern is directly borrowed from CPU L2 cache pre-fetching and database read-ahead buffering. Both fields solved the reactive-retrieval latency problem decades ago. Applying it to agent memory retrieval is novel because no agent memory framework has modeled session context as a *predictable access pattern* — they have all treated each retrieval call as an independent random event.

**Benchmark:** Compare P95 retrieval latency: cold retrieval vs. pre-fetch cache hit. Also report pre-fetch hit rate across 100 test sessions. Target: >60% hit rate, >40% P95 latency reduction.

## 5.4 Cross-Session Confidence Accumulation

### The Gap

Mem0 assigns a static confidence score at extraction time and never changes it. No tool models the epistemic question: *how confident should the agent be in this belief after seeing corroborating or*

contradicting evidence across multiple sessions? This matters because an agent that treats a one-time statement at the same confidence as a fact confirmed across 12 sessions will behave unreliably.

The Confidence Update Rules

| Event                                              | Effect on Confidence                                     | Notes                                               |
|----------------------------------------------------|----------------------------------------------------------|-----------------------------------------------------|
| Corroboration — same fact appears in a new session | Confidence increases; returns after ~5 corroborations    | Prevents runaway confidence inflation               |
| Contradiction — conflicting fact appears           | $-\Delta$ proportional to contradiction's own confidence | Feeds into conflict detection engine                |
| Dormancy — unreferenced for N sessions             | Slow downward drift                                      | Distinct from decay — this affects trustworthiness  |
| Crystallization — confidence exceeds 0.95          | Decay paused; confidence drift stopped                   | The agent now 'knows' this fact with high certainty |

Decay vs. Confidence — A Critical Distinction

These are two orthogonal axes. **Decay** controls retrieval priority — a high-confidence, decayed memory may not surface in search results because it is old. **Confidence** controls how much the agent should trust the memory once retrieved — a low-confidence, recently retrieved memory may appear in context but should be hedged in the agent's response. Both axes are independently meaningful and independently queryable.

**Benchmark:** Confidence calibration — verify that a memory at confidence=0.8 is empirically correct approximately 80% of the time when tested against a ground-truth dataset of user facts. Report calibration error (ECE score).

5.5 Cross-Agent Memory Scoping — Permission Layer

The Gap

In multi-agent systems built with LangGraph or CrewAI, each agent has its own memory and there is no persistent cross-session cross-agent permission model. LangGraph has shared state within a single graph execution, but this state does not persist between runs. CrewAI's memory=True flag stores to a local SQLite — not queryable by a different agent in a different workflow. When Agent A learns something about the user, Agent B in the same organization has no formal mechanism to access it.

The Four Scope Levels

| Scope      | Access Rule                                                              | Primary Use Case                                           |
|------------|--------------------------------------------------------------------------|------------------------------------------------------------|
| private    | Read/write: originating agent only                                       | Agent-specific working knowledge not relevant to others    |
| team       | Read/write: all agents in a defined group (e.g., one LangGraph workflow) | Shared context within a single product's agent pipeline    |
| global     | Read: any agent in the deployment; Write: orchestrator only              | Organization-wide user facts (name, timezone, preferences) |
| user-owned | Read: user + any agent; Delete: user only                                | GDPR-visible memories the user can inspect and delete      |

LangGraph Integration Example

```
# Orchestrator agent promotes a memory to team scope
await recall.promote(
  memory_id='mem_0291',
  scope='team',
  team_id='logistics_pipeline_v2'
)
```



```
# Downstream Validator agent retrieves it without a separate ingestion call
ctx = await recall.retrieve(
    query='user deployment preferences',
    token_budget=800,
    scope='team',
    team_id='logistics_pipeline_v2'
)
```

GDPR Compliance Story

The user-owned scope doubles as a GDPR transparency mechanism. A single API call GET /memory/user-owned returns every memory the system holds that the user is entitled to see and delete. No other memory product has a formalized user-visible memory tier with a one-call inspection interface.

**Benchmark:** In a 3-agent LangGraph pipeline, measure redundant retrieval calls with siloed memory vs. team-scoped memory. Target: >50% reduction in duplicate retrieval calls across agent boundaries.

5.6 Priority Build Recommendation

Given the 9 July deadline and a three-week build window, not all five features can be built to production quality. The following prioritization is based on three criteria: uniqueness in the market, narrative clarity for a 3-minute demo, and implementation feasibility.

| Priority             | Feature                               | Why This Order                                                                                     | Week        |
|----------------------|---------------------------------------|----------------------------------------------------------------------------------------------------|-------------|
| P1 — MUST BUILD      | Conflict Detection + Provenance Graph | These two compose naturally — conflict resolution automatically writes into the provenance graph.  | Week 1      |
| P2 — STRONG ADDITION | Proactive Pre-fetching                | Implementable in ~4 days. Produces a compelling latency benchmark chart. The systems-c             | Week 2      |
| P3 — IF TIME PERMITS | Confidence Accumulation               | Adds epistemic depth to every memory unit. Composes with Week 1 features naturally. Th             | Week 3      |
| P4 — FUTURE ROADMAP  | DMAP-Agent Scoping                    | High architectural value but requires multi-agent test harness. Scope is difficult to stub the API | Post-launch |

The single clearest competitive claim Recall can make:

Every other memory tool is a library. Recall is an epistemic system. It does not just store what the agent knows — it models how confident the agent should be, tracks the evidence behind every belief, detects when beliefs contradict each other, and forgets what is no longer relevant. That is a fundamentally different category of product.